

Simple Programming Language using Lex and Yacc

Number of Students per Group

3-4 Students

Overview

It is required to implement a compiler for a subset of the C++ programming language using the Lex and Yacc compiler generating packages.

Requirements

- Implement a subset of the C++ programming language. The important constructs to be considered are:
 - Standard C++ data types.
 - Variables and Constants declaration.
 - Mathematical and logical expressions.
 - Assignment statements.
 - If-then-else statement, while loops, do-while loops, for loops, switch statement (matching standard C++ behavior including break fall-through).
 - Block structure (nested scopes following C++ rules, including inline declarations like `for(int i=0; i<10; i++)` and chained declarations like `int x = 5, y, z = 10;`).
 - Functions (Must handle parameters, default values, and recursion).
- Design a suitable and extensible format for the symbol table.
- Implement the lexical analyzer using Lex.
- Design suitable action rules to produce the output quadruples in the exact format (Operator, Arg1, Arg2, Result) and implement your parser using YACC.
- Implement a proper syntax error handler using Error Recovery (the compiler should attempt to find multiple errors rather than halting on the first syntax error).
- Build a simple semantic analyzer that checks for the following:
 - Variable declaration conflicts. i.e. multiple declarations of the same variable.
 - Improper usage of variables regarding their type.
 - Variables used before being initialised and unused variables.
 - Implicit type conversion quadruples to match C++ operators' semantic requirements (i.e. converting integer to real, etc.).
 - The left-hand side of an assignment must be valid

- Constants must be initialized upon declaration and cannot be reassigned.
- Function Validation and Control Flow Semantics (break, continue, return, ..)
- **[Optional]** Implement a simple GUI.

Project Phases

- **Phase I:** In this phase, you're required to deliver your lex and yacc files i.e, your lexer and parser.
- **Phase II:** In this phase, you're required to modify your implementations to include the following:
 - Design a suitable and extensible format for the symbol table.
 - Design suitable action rules to produce the output quadruples.
 - Implement a proper syntax error handler.
 - Build a simple semantic analyzer.

Deliverables

- Source code of your project.
- A Document that contains the following:
 - Project Overview
 - Tools and Technologies used
 - A list of tokens and a description of each
 - A list of the quadruples and a short description of each e.g:

Quadruple	Description
(JMP, -, -, L)	Unconditional jump to label l
(ADD, V1, V2, V3)	$V3 = V2 + V3$

Program evaluation

- The program is to be fed by a source code file containing your language and do the following:
 - Produce the corresponding quadruples.
 - Display syntax errors that exist in your program.
 - Display the semantic errors that exist in your program.
 - Display the symbol table.

Evaluation Criteria

- The correctness of your quadruples.
- Handling syntax and semantic errors.
- Teamwork and good documentation.